

Zastosowanie algorytmu A^* do znajdowania ścieżki w labiryncie

Celem projektu była implementacja algorytmu A^* oraz wykorzystanie go do znajdowania najkrótszej ścieżki pomiędzy punktem startowym a punktem końcowym w labiryncie reprezentowanym jako siatka dwuwymiarowa.

Dodatkowo przeprowadzono porównanie działania algorytmu dla różnych heurystyk oraz analizę uzyskanych wyników.

Opis problemu

Problem polega na znalezieniu najkrótszej możliwej trasy od punktu startowego do punktu końcowego przy omijaniu przeszkód.

Labirynt został przedstawiony jako siatka 2D, gdzie:

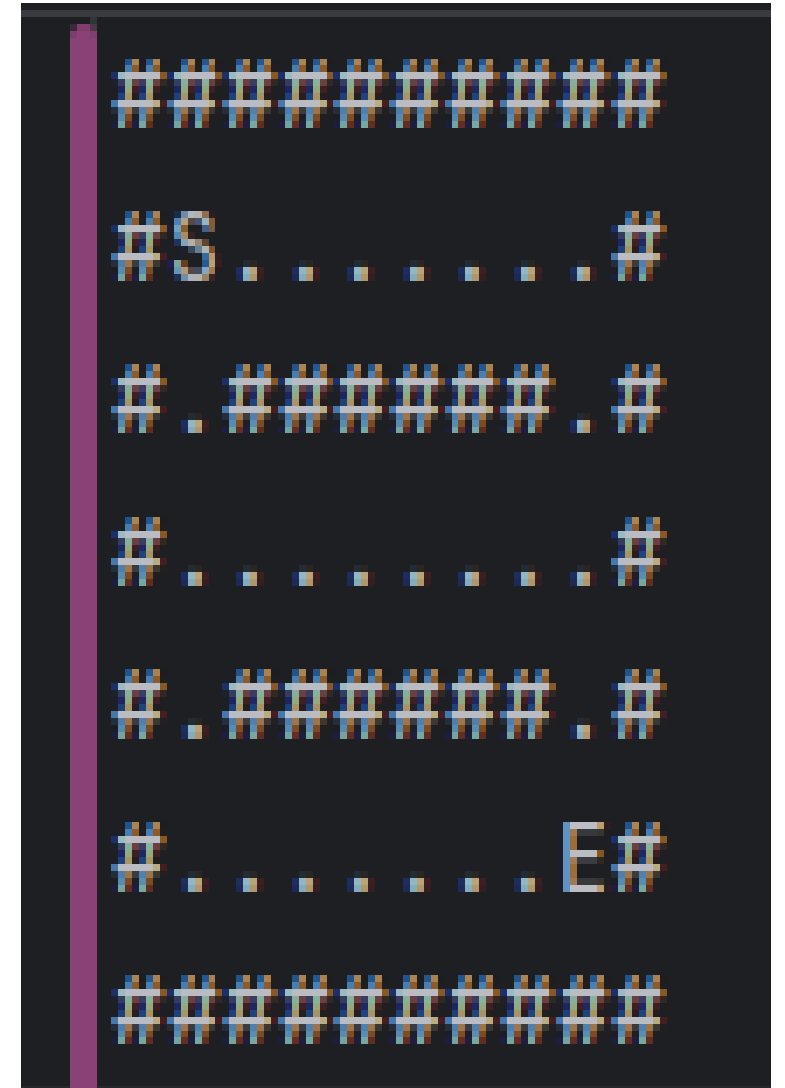
- S oznacza punkt startowy,
- E oznacza punkt końcowy,
- oznacza ścianę,
- . oznacza pole, po którym można się poruszać.

Agent może poruszać się tylko w czterech kierunkach: góra, dół, lewo i prawo.

Opis danych

- Danymi wejściowymi są pliki tekstowe zawierające mapy labiryntów.
- Każdy labirynt jest zapisany jako układ znaków reprezentujących ściany, wolne pola, punkt startowy oraz punkt końcowy.

W projekcie wykorzystano kilka labiryntów o różnym stopniu złożoności, aby sprawdzić zachowanie algorytmu w różnych warunkach.



Uzasadnienie wyboru algorytmu

- Do rozwiązania problemu wybrano algorytm A*, ponieważ jest jednym z najpopularniejszych algorytmów wyszukiwania ścieżki.
- Jego zaletą jest wykorzystanie heurystyki, która pozwala przewidywać kierunek, w którym znajduje się cel. Dzięki temu algorytm nie musi analizować wszystkich możliwych ścieżek i zwykle działa znacznie szybciej od metod takich jak BFS czy Dijkstra.
- Algorytm A* gwarantuje znalezienie najkrótszej ścieżki przy odpowiednio dobranej heurystyce.

Działanie algorytmu A*

- Algorytm A* dla każdego pola oblicza wartość:

- $f(n)=g(n)+h(n)$

gdzie:

- $g(n)$ oznacza koszt dotarcia od startu do aktualnego pola,
- $h(n)$ oznacza szacowaną odległość od aktualnego pola do celu.

Algorytm zawsze wybiera pole o najniższym koszcie całkowitym i sprawdza jego sąsiadów. Proces jest powtarzany aż do odnalezienia punktu końcowego.

Heurystyki

Heurystyka jest funkcją, która szacuje, jak daleko od celu znajduje się aktualne pole.

Heurystyka Manhattan

- Oblicza odległość jako sumę różnic w pionie i poziomie.
- Jeżeli cel znajduje się 3 pola w prawo i 2 pola w dół, heurystyka zwróci wartość 5.

Heurystyka Euklidesowa

- Oblicza odległość w linii prostej między punktami.

Zerowa heurystyka

- Zawsze zwraca wartość 0, działa wtedy podobnie do algorytmu Dijkstry.

Projekt został podzielony na kilka modułów:

- `astar.py` - implementacja algorytmu A^* ,
- `heuristics.py` - implementacja heurystyk,
- `maze.py` - wczytywanie labiryntów,
- `visualization.py` - prezentacja wyników,
- `main.py` - uruchamianie testów i pomiarów.

Dodatkowo przygotowano testy jednostkowe sprawdzające poprawność działania heurystyk oraz algorytmu.

Wyniki i wnioski

- Algorytm A* poprawnie znajdował najkrótszą ścieżkę we wszystkich testowanych labiryntach.
- Najlepsze wyniki uzyskano dla heurystyki Manhattan, ponieważ odpowiada ona sposobowi poruszania się po labiryncie.
- Heurystyka Euklidesowa również znajdowała poprawne rozwiązania, ale zwykle analizowała większą liczbę pól.
- Najgorsze wyniki uzyskano dla zerowej heurystyki, która nie dostarczała algorytmowi żadnej informacji o położeniu celu.

Projekt pokazał, że odpowiednio dobrana heurystyka może znacząco przyspieszyć działanie algorytmu wyszukiwania ścieżki.